

AIM : Numerical Computation with NumPy: Perform different computations using NumPy library.

Theory :

1. Explain the role of NumPy in the Python scientific computing ecosystem?

Ans: NumPy serves as the high-performance engine of the Python data science world, providing the essential infrastructure needed to manage and manipulate massive, multi-dimensional data structures. By introducing specialized arrays and matrices that are far more memory-efficient than standard Python lists, it allows researchers and developers to execute complex mathematical operations with incredible speed. It effectively acts as the structural bedrock for the entire ecosystem, powering household names like Pandas and Scikit-learn to handle everything from financial modeling to artificial intelligence. Ultimately, NumPy transforms Python from a general-purpose language into a powerhouse for scientific computing, making it an indispensable tool for anyone analyzing data or running sophisticated simulations in both academic and industrial settings.

2. Why is NumPy considered faster than pure Python computations?

Ans: NumPy achieves its remarkable speed by bypassing the inherent slowness of standard Python loops, instead relying on highly optimized C and C++ code under the hood. While a typical Python list stores objects scattered across memory, NumPy organizes data in contiguous blocks, allowing the computer's CPU to access and process information much faster. The real "secret sauce," however, is vectorization, which enables the library to perform a single mathematical operation across an entire dataset simultaneously rather than iterating through items one by one. This combination of hardware-level memory management and the elimination of explicit loops makes NumPy the gold standard for high-speed data processing and complex scientific calculations.

3. What types of mathematical operations can be performed using NumPy?

Ans: NumPy supports a wide variety of mathematical operations, including basic arithmetic operations such as addition, subtraction, multiplication, and division. It also provides advanced mathematical functions like trigonometric operations (sin, cos), logarithmic functions, exponentiation, and statistical operations such as mean, median, and standard deviation. NumPy also supports linear algebra operations including matrix multiplication, determinant calculation, and eigenvalues. These capabilities make NumPy a powerful tool for performing both simple and complex numerical computations efficiently.

4. Explain indexing and slicing in NumPy arrays?

Ans: Indexing and slicing in NumPy arrays allow users to access and manipulate specific elements or subsets of data efficiently. Indexing refers to accessing a single element using its position, while slicing refers to extracting a portion of the array using a range of indices. NumPy supports both one-dimensional and multi-dimensional indexing, making it easy to work with complex data structures. Slicing is especially useful when working with large datasets, as it allows users to select only the required portion without copying the entire array.

5. How does NumPy integrate with libraries like Pandas, Matplotlib, and Scikit learn?

Ans: NumPy integrates seamlessly with other popular Python libraries such as Pandas, Matplotlib, and Scikit-learn, making it a core component of the data science ecosystem. Pandas uses NumPy arrays as its underlying data structure, enabling efficient data manipulation and analysis. Matplotlib relies on NumPy arrays for plotting graphs and visualizing numerical data. Scikit-learn, a machine learning library, uses NumPy arrays to handle datasets and perform computations for training models. This integration allows smooth data flow between libraries and enhances performance and flexibility in data science workflows.

Import NumPy

```
import numpy as np
```

NmPy vs Python Speed Comparson

This cell compares the execution speed of pure Python and NumPy. First, a Python list is created and each element is multiplied by 2 using a loop-like operation (list comprehension). Then, a NumPy array is created and multiplied by 2 using vectorized operations. The time module is used to measure the execution time for both approaches. This demonstrates that NumPy is faster because it performs operations at a lower level using optimized code and avoids explicit loops.

```
import time
import numpy as np
# Python list computation
start = time.time()
lst = list(range(1000000))
lst = [x * 2 for x in lst]
end = time.time()
print("Python Time:", end - start)
# NumPy computation
start = time.time()
arr = np.arange(1000000)
arr = arr * 2
```

```
end = time.time()
print("NumPy Time:", end - start)
```

```
Python Time: 0.16419529914855957
NumPy Time: 0.011341094970703125
```

Basic Mathematical Operations

In this cell, a NumPy array is created and basic arithmetic operations such as addition, subtraction, multiplication, and division are performed on it. These operations are applied element-wise, meaning each element in the array is processed individually. This shows how NumPy simplifies mathematical computations and allows operations to be performed on entire datasets efficiently.

```
arr = np.array([5, 10, 15, 20, 25])
print("Original Array:", arr)
print("Addition:", arr + 2)
print("Subtraction:", arr - 1)
print("Multiplication:", arr * 3)
print("Division:", arr / 2)
```

```
Original Array: [ 5 10 15 20 25]
Addition: [ 7 12 17 22 27]
Subtraction: [ 4  9 14 19 24]
Multiplication: [15 30 45 60 75]
Division: [ 2.5  5.  7.5 10. 12.5]
```

Advanced Mathematical Functions

This cell demonstrates advanced mathematical and statistical functions provided by NumPy. Functions like `sqrt()` calculate square roots, `power()` raises elements to a specified power, and statistical functions like `mean()`, `median()`, and `std()` compute important measures of central tendency and dispersion. These built-in functions make NumPy highly useful for data analysis and scientific computing.

```
print("Square Root:", np.sqrt(arr))
print("Power:", np.power(arr, 2))
print("Mean:", np.mean(arr))
print("Median:", np.median(arr))
print("Standard Deviation:", np.std(arr))
```

```
Square Root: [2.23606798 3.16227766 3.87298335 4.47213595 5.          ]
Power: [ 25 100 225 400 625]
Mean: 15.0
Median: 15.0
Standard Deviation: 7.0710678118654755
```

Indexing and Slicing (1D Array)

This cell explains how to access elements in a one-dimensional NumPy array using indexing and slicing. Indexing is used to retrieve a specific element by its position, while slicing is used to extract a subset of elements using a range. For example, `arr[1:4]` returns elements from index 1 to 3. This helps in efficiently working with subsets of data.

```
arr = np.array([15, 20, 25, 30, 35])
print("First Element:", arr[0])
print("Last Element:", arr[-1])
print("Slice (1 to 3):", arr[1:4])
```

```
First Element: 15
Last Element: 35
Slice (1 to 3): [20 25 30]
```

Indexing in 2D Array

In this cell, a two-dimensional NumPy array (matrix) is created, and different indexing techniques are demonstrated. Elements can be accessed using row and column indices, such as `arr2[1, 2]`. Additionally, entire rows or columns can be extracted using slicing. This is useful when working with structured data like tables or matrices

```
arr2 = np.array([[5, 10, 15],
                 [20, 25, 30]])

print("Element at row 1, col 2:", arr2[1, 2])
print("First Row:", arr2[0])
print("Second Column:", arr2[:, 1])
```

```
Element at row 1, col 2: 30
First Row: [ 5 10 15]
Second Column: [10 25]
```

NumPy with Pandas

This cell shows how NumPy integrates with the Pandas library. A NumPy array is created and then converted into a Pandas DataFrame. Pandas uses NumPy arrays internally, making data manipulation efficient. This integration allows users to easily move data between NumPy and Pandas for analysis and processing.

```
import pandas as pd
data = np.array([100, 200, 300, 400, 500])

df = pd.DataFrame(data, columns=["Values"])
print(df)
```

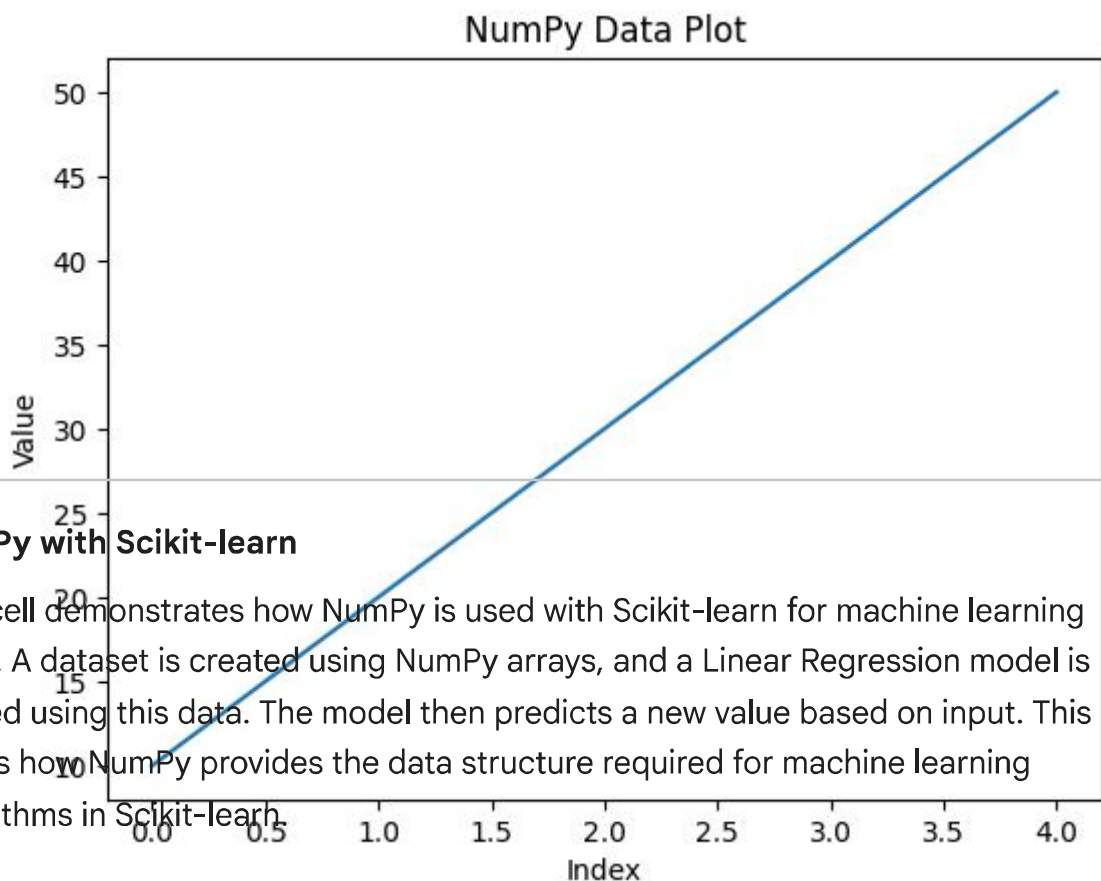
```
   Values
0     100
1     200
```

2	300
3	400
4	500

NumPy with Matplotlib

In this cell, a NumPy array is used to create a simple line plot using Matplotlib. The array values are plofled on a graph, and labels are added for befler visualization. This demonstrates how NumPy works with Matplotlib to visualize numerical data, which is important for data analysis and interpretation.

```
data = np.array([10, 20, 30, 40, 50])
import matplotlib.pyplot as plt
data = np.array([10, 20, 30, 40, 50])
plt.plot(data)
plt.title("NumPy Data Plot")
plt.xlabel("Index")
plt.ylabel("Value")
plt.show()
```



NumPy with Scikit-learn

This cell demonstrates how NumPy is used with Scikit-learn for machine learning tasks. A dataset is created using NumPy arrays, and a Linear Regression model is trained using this data. The model then predicts a new value based on input. This shows how NumPy provides the data structure required for machine learning algorithms in Scikit-learn.

```
from sklearn.linear_model import LinearRegression
# Sample data
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])
# Model
model = LinearRegression()
model.fit(X, y)
# Prediction
```

```
prediction = model.predict([[6]])  
print("Prediction for 6:", prediction)
```

```
Prediction for 6: [12.]
```